

# Geoguessr Geolocation

## Team Members:

- Tai Nguyen; Email: `taing@seas.upenn.edu`
- Josh Ludan; Email: `jludan@seas.upenn.edu`

home pod: lovable-lemur (Josh), Friday 4-5pm (Tai)

---

## Abstract

In this paper, we tackle the problem of image geolocation in the domain of the game Geoguessr. Our goal is to train a model that can predict the location of an image taken by a Google Street View car. We improve upon previous work by utilizing more efficient models, incorporating pretraining and using additional datasets. Our baseline model uses Resnet50 and KNN, and we experiment with EfficientNet and Swin Transformer. Our best model achieves an accuracy of 54.2%, significantly outperforming the previous state-of-the-art model, which only used Resnet50 and had an accuracy of 25.92%. Our experiments reveal that the main improvements come primarily from pretraining on imagenet, followed by improved model architecture.

## 1 Motivation

GeoGuessr, created in 2013, is a web-based game that challenges its players to guess the location on the map given a Google Street view image of that location. Each guess is bounded by a time limit and can be played solo or in multiplayer mode. For our project, we want to build a deep learning model that can accurately classify the location of a given image. Given the strong pattern recognition capabilities of these models, we think that they stand a good chance of being competitive players for this game.

## 2 Related Work

There are several works we can draw influence from in building our model. These works can be broken down into works that perform general geolocation and geolocation in the GeoGuessr setting. The works that have tried to solve these problems are quite diverse, with some utilizing classical machine learning approaches and with some using deep learning.

The specific task of creating a model that can perform well at GeoGuessr can be thought of as a subproblem within the specific field of image geolocation (predicting the location of an image). The earliest paper we found tackling this problem was published by Hays et al. in 2008 [5] titled "IM2GPS: estimating geographic information from a single image". In this paper, they utilized similarity-based approaches such as KNN to classify an image dataset with accompany location data. This work has been expanded upon in recent years with more recent papers utilizing deep learning approaches [14] [9] [10]. These more recent papers commonly utilize CNNs, with the most recent paper from Pramanick et al. published in 2022 utilizing transformers. These papers can help serve as inspiration in building an optimal model.

Beyond these papers which perform the geolocation task type, we also have multiple sources which perform the geolocation in the context of Geoguessr. One key difference between the models trained for Geoguessr and models trained for general geolocation is that the distribution of Geoguessr images are much more specialized. Geoguessr images come from Google streetview images which mean that they tend to be outdoors, in daylight, and on the street. Compare this to images from IM2GPS which come with none of those guarantees.

The sources for these Geoguessr models span from YouTube videos [2] to student projects [3] [13], blog posts [7], and even to published papers [11]. These sources commonly structure the Geoguessr task as a classification task localized to predicting which state a photo was location was taken. They often use CNNs as the foundation of their model, with some papers incorporating some degree of finetuning on pretrained models such as VGG.

Digging deeper into the most relevant paper for our task, Suresh et al. [11] provides a dataset named **50States10K**, containing half a million web-scraped images of US locations. They also trains a neural network on a 50-layer ResNet-based architecture that achieves a 38.32% in classifying US states. This work provides both a large dataset and a baseline that our model could benchmark against.

In their architecture, DeepGeo experiments with different variations of a 50-layer Resnet. The models differ in the integration method for 4 different views of the same location throughout the CNN: 1) Concatenation at the beginning of the network, 2) Concatenation in the end (through a single layer perceptron), and 3) Maxpooling over the 4 network outputs and take prediction as one with the highest confidence. The second approach was the highest performer at 38% accuracy. The main weakness of DeepGeo model is a tendency to misclassify locations where the population is not as densely populated. They explain this as reflective of their sampling techniques, which eliminates sampled location where "population density is below a threshold  $p_{min}$ ". In this regard, perhaps data augmentation can help make up for the under representation of these (mostly rural) areas.

Other sources of data are available in the context of GeoGuessr. A Kaggle repository [1] also includes data collected from GeoGuessr at a country-level. We plan to combine the images from this repository with "United States" label to increase the size of our train and test sets.

## 3 Dataset

### 3.1 Sources

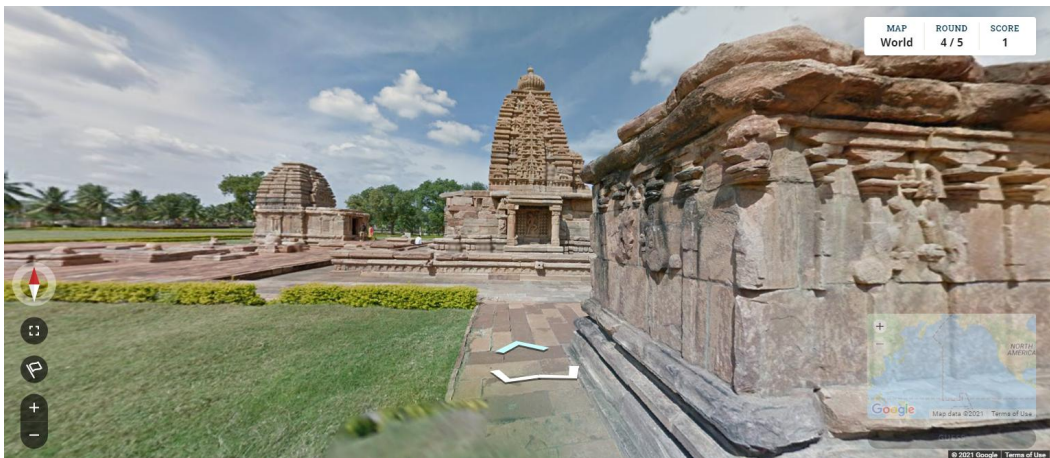


Figure 1: From GeoGuessr dataset, a sample image of India containing the Geoguessr HUD.

We rely on two datasets for this task. The first is scrape of Geoguessr itself from Kaggle [1] and the second is from a paper by Suresh et al. [11] which scraped google streetview itself using Google’s Street Siew API [4].

The first scrape contains approximately 50,000 images from 124 different countries. This scrape notably did not use the Google Street View API and instead scraped Geoguessr itself for images. The second scrape contains 500,000 images, or 10,000 images from each US state. Both scrapes contain accompanying location data. In the first scrape, this is limited to country-level location while in the second scrape we have accompanying coordinate level information alongside state level information. We also note some important differences between the two datasets. The first scrape which contains images by country have images which are high-resolution, wide-aspect ratio, and contain HUD elements (navigation bars, etc.) from the Geoguessr interface. The second scrape which deals with US States, in contrast, contains lower resolution square images



Figure 2: From DeepGeo dataset, side-by-side views of a highway in Arizona taken from  $[0^\circ, 90^\circ, 180^\circ, 270^\circ]$  angles. Each input is a set of 4 images with Google Street watermark.

with Google Street View watermarks. As an observation, while evaluating the model trained on both datasets, we notice that there is little confusion about which distribution the model thinks the test image came from. We will discuss this aspect further in the Methods section.

### 3.2 Sampling

Since the Geoguessr dataset is heavily *imbalanced* with many underrepresented countries, we filtered the data to exclude countries with less than 100 images entirely.

Additionally, since we are evaluating our model with a single-image approach, we ensure that all 4 images for a given location are included in the same split. This helps us avoid potential *data leakage*, where a model could see the  $0^\circ$  image of Arizona during training and the  $270^\circ$  of the same shot during testing, making it possibly trivial for the model to predict. Together, the combination of the two datasets gives us 12.3 GB worth of images on 106 US state and country labels.

## 4 Problem Formulation

We structure our problem as a classification task: predicting the location label (either state or country) of given a single input image.

## 5 Methods

### 5.1 Single-image

While previous works have attempted to integrate different cardinal directions, such as a concatenating 4 views into a single image or integrating them in a neural network [11], we choose the single-image approach for training and evaluation. The model sees just an image at run time and outputs a prediction. This is preferred for two reasons: 1) The country images only have a single view and 2) It allows for the model to be used in more general settings

### 5.2 Baselines

For our first baseline, inspired by [5], we utilize KNN on the raw 25x25 pixels. We resize our image using cv2's bilinear interpolation to 25x25 and flatten it and use the resulting 625-dimensional vector as an input to the KNN. Ideally, the model would create a baseline that is better than random and an interpretable clustering of street view images. We note that using a baseline of guessing the majority class (South Carolina) would allow the model to obtain an accuracy of 2.18%

### 5.3 Convolutional Neural Nets

We choose two different CNN models as the baselines: Residual Network (ResNet) [6] and Efficient Net [12]. In both of these cases, we used the pretrained weights of these models trained on ImageNet to finetune them further on our dataset. In finetuning, there are a few approaches for repurposing a pretrained model: 1) Update the weights of the entire network, 2) Update some middle layers and leave the others frozen and 3) Update just the last layer used for classification. In our models, we choose approach #1, which we observed to help achieve the highest performance compare to purely updating the last layer.

#### 5.3.1 ResNet

ResNet [6] uses a concept called "residual blocks" that enables neural networks at the time to be trained for much deeper than they were, achieving accurate results on the depth representation. There are many variations of the architecture that differs by the number of layers in the network. For our model, we chose ResNet with 50 layers to serve as a good baseline since this is also the architecture chosen by the authors of DeepGeo [11].

#### 5.3.2 EfficientNet

Before EfficientNet, neural networks scaled up in a rigid manner to achieve performance gains. In the case of ResNet, for example, this simply means adding more layers to the network until the return on investment vanishes, without any regard to computational resources. EfficientNet [12] uses a novel *compound coefficient* scaling method that achieves better accuracy while being 10 times more efficient. To optimize training resource constraints, we use the pretrained EfficientNet B0, the smallest network in the EfficientNet family, to finetune further on our dataset. This model is our most competitive baseline.

### 5.4 Transformer

In the past year and a half, architectures using Transformer has produced many state-of-the-art results for many computer vision tasks. One such example is Swin Transformer [8], an architecture that combines image patches in deep layers of the network to produce a rich representation. Similar to other CNNs, we use the pretrained weights of ImageNet-1K trained on Swin Transformer Tiny and finetune the last head attention layer for our dataset. Due to limited computation resources, finetuning is only done for 3 epochs for the Swin model.

### 5.5 Data Augmentation

For this model, we decided to make the choice to be nonaggressive during image augmentation, opting only to use random cropping. This is due to the fact that even slight distortions such as skew, hue adjustments, or flipping can destroy useful information. Looking at Geoguessr site guides such as geotips.net, we see that human players use information like the camera distortion and which side of the road cars are on as key indicators. Adding skew or flips would destroy these markers, which is why we decided not to do them.

## 6 Experiments and Results

### 6.1 All baselines

| Model                 | Size  | Epoch | Accuracy |       |       |
|-----------------------|-------|-------|----------|-------|-------|
|                       |       |       | Top-1    | Top-3 | Top-5 |
| DeepGeo*              | -     | 50    | 25.92    | 46.51 | 58.25 |
| KNN                   | -     | -     | 14.24    | -     | -     |
| ResNet 50             | 25.6M | 8     | 44.17    | 66.75 | 77.0  |
| EfficientNet B0       | 5.3M  | 8     | 54.2     | 76.1  | 84.4  |
| Swin Transformer Tiny | 28.3M | 3     | 30.8     | 52.7  | 64.3  |

Table 1: Results evaluated by top-k predictions.

\*DeepGeo is only evaluated on U.S states dataset.

From the table, we see that the EfficientNetB0 model outperforms all the other models by a wide margin. We also note that both Resnet and EfficientNet outperform DeepGeo. This is especially remarkable given the fact that DeepGeo effectively has access to 4 times as much information since it uses 4 images to make a prediction. This is also interesting because Resnet 50 should ideally have the similar results as DeepGeo since DeepGeo is based on Resnet. We suspect that this discrepancy is largely due to the lack of Pretraining in DeepGeo. We investigate this further in section 7.2. The KNN performs remarkably well, achieving a 14.24% accuracy despite the fact that it uses essentially no feature engineering.

## 7 Discussion

### 7.1 Multi-task vs. Single task setting

| Model                       | Size  | Epoch | Top-1 Accuracy |           |          |
|-----------------------------|-------|-------|----------------|-----------|----------|
|                             |       |       | States         | Countries | Combined |
| ResNet 50 (Both)            | 25.6M | 3     | 32.22          | 25.93     | 32.07    |
| ResNet 50 (States)          | 25.6M | 3     | 31.33          | -         | -        |
| ResNet 50 (Countries)       | 25.6M | 3     | -              | 46.67     | -        |
| EfficientNet B0 (Both)      | 5.3M  | 3     | 46.3           | 57.3      | 47.3     |
| EfficientNet B0 (States)    | 5.3M  | 3     | 46.0           | -         | -        |
| EfficientNet B0 (Countries) | 5.3M  | 3     | -              | 55.7      | -        |

Table 2: Top-1 evaluation; Single-task settings and multi-task setting

From the table, we see that in EfficientNet, learning on the countries datasets helps performance on the state prediction task and learning on the states dataset helps performance on the country prediction task. This is especially apparent in the case of the improvement for country prediction. Including the state dataset increases the prediction accuracy by 0.3% on the country dataset for EfficientNet and including the state dataset increases the prediction accuracy on the country dataset by 1.6% for EfficientNet. This performance increase is likely due to the fact that the model can take advantage of the feature learning in one domain

and apply it to another. The fact that we see more of an increase in the country dataset is likely due to the fact that the state dataset is 10 times bigger than the country dataset.

With ResNet, we see an increase performance for U.S states when learning from countries but not vice versa. In fact, for countries, the performance dropped when model leans multi-task. This could be due to an architectural difference between the two model architectures or the fact that ResNet is much more inefficient, thus requires more train epochs to reap the benefits.

## 7.2 Impact of transfer learning

| ResNet 50 Initialization                  | Epoch | Accuracy |       |       |
|-------------------------------------------|-------|----------|-------|-------|
|                                           |       | Top-1    | Top-3 | Top-5 |
| DeepGeo                                   | 50    | 25.92    | 46.51 | 58.25 |
| EfficientNet B0 Random initialization     | 3     | 29.01    | 51.00 | 63.21 |
| EfficientNet B0 with Imagenet Pretraining | 3     | 47.3     | 70.1  | 79.9  |

Table 3: Compare different training settings of ResNet 50-layer.  
\*DeepGeo is only evaluated on U.S states dataset.

From experiments in Table 4, the benefits of using *pre-trained* models are clear and stark. A ResNet50 pretrained on a large benchmark (ImageNet) and similar task (image classification) achieves almost a 11% difference compared to a ResNet50 model trained with random initializations. The effect can be seen especially when comparing with DeepGeo, which was trained for 16 times longer at 50 epochs. Recall that our train and evaluation sets are supersets of DeepGeo, which explains why the accuracies are boosted by country labels and thus better across.

Our experiments verify the power of the pretraining and finetuning paradigm as a contrast to training from scratch in computer vision.

## 7.3 Impact of convergence

We note that in the Resnet, Efficientnet, and Swin transformer models that we made, the training accuracy was essentially the same as the testing accuracy in the highest epochs we tested, indicating that we have not fully fit the model and additional epochs can still increase accuracy. We decided to settle on 8 epochs for Resnet and Efficientnet and 3 for the Swin transformer model due to resource constraints, however we note that these models still have some room for improvement just by increasing the training time.

## 7.4 By-class accuracy for best model

From Figure 3, the top 5 performing classes are Hawaii, Alaska, Japan, United States, and South Africa. On the other hand, our model sees 0% accuracy on 9 countries: Latvia, Portugal, Ukraine, Denmark, Czechia, Croatia, Cambodia, Slovakia, and Belgium. By the nature of our combined datasets, countries have less training data than states. Thus, it is reasonable to expect that the worst-performing categories are all countries as reflected by the Accuracy Chart. Something interesting to note here is that the Hawaii and Alaska may be getting high accuracies due to the fact that they are separated from the mainland US and thus are less likely to be confused with other states. In addition to this, we see that states on the west coast tend to have a higher accuracy compared to states on the east coast. This is likely due to the fact that these states are bigger and contain distinctive climate and land features not seen in other states. The small and compact nature of eastern states may lead to these states being more easily confused with each other.

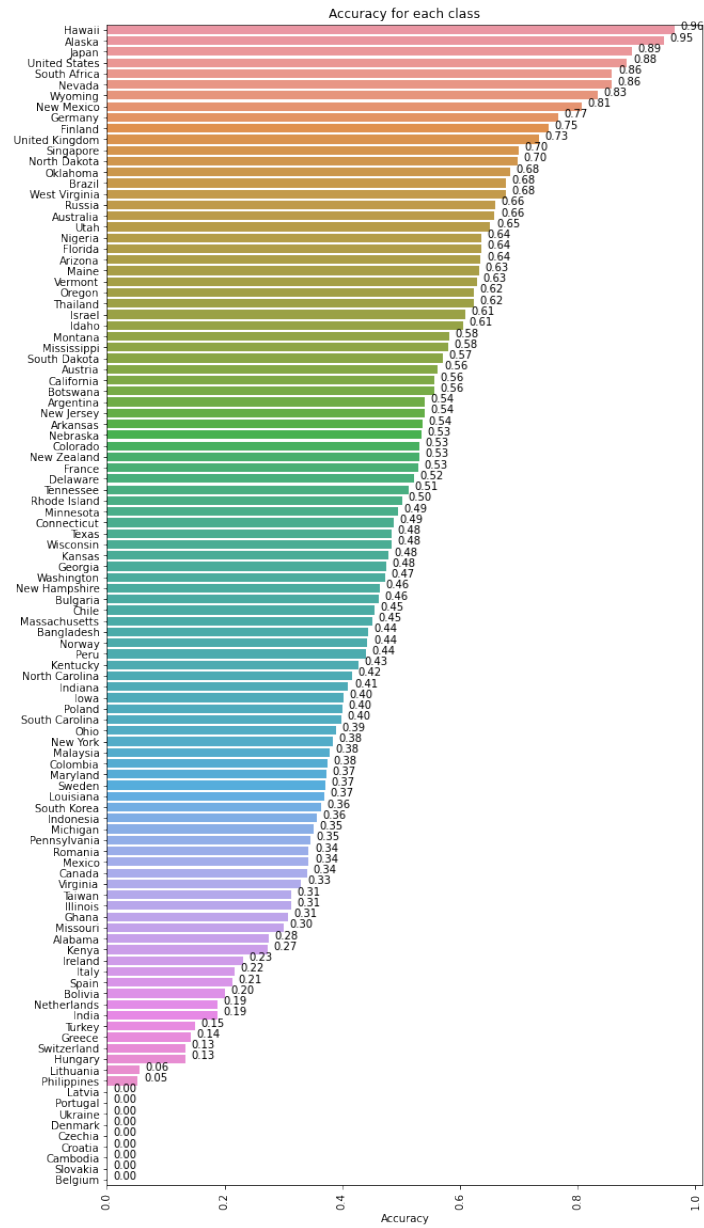


Figure 3: Accuracy bar chart per class

|                | Number of items | Accuracy           | Top 3 most confused                  | Top 3 most confused % |
|----------------|-----------------|--------------------|--------------------------------------|-----------------------|
| Hawaii         | 984             | 0.9644308943089430 | Louisiana, Pennsylvania, Nevada      | 0.41%, 0.41%, 0.3%    |
| Alaska         | 952             | 0.9464285714285710 | Utah, Maine, Hawaii                  | 0.95%, 0.84%, 0.63%   |
| Japan          | 409             | 0.8924205378973110 | Thailand, France, United States      | 2.44%, 1.71%, 0.73%   |
| United States  | 1188            | 0.8838383838383840 | Brazil, Canada, France               | 1.68%, 1.43%, 1.35%   |
| South Africa   | 120             | 0.8583333333333330 | Russia, Kenya, Australia             | 5.0%, 1.67%, 1.67%    |
| Nevada         | 1076            | 0.8578066914498140 | Utah, Wyoming, New Mexico            | 6.51%, 2.04%, 1.12%   |
| Wyoming        | 992             | 0.8336693548387100 | South Dakota, Idaho, Colorado        | 2.52%, 2.42%, 1.51%   |
| New Mexico     | 984             | 0.806910569105691  | Arizona, Oklahoma, Nevada            | 5.49%, 2.03%, 2.03%   |
| Germany        | 77              | 0.7662337662337660 | Japan, France, United Kingdom        | 6.49%, 5.19%, 5.19%   |
| Finland        | 116             | 0.75               | United States, Russia, Canada        | 5.17%, 4.31%, 3.45%   |
| United Kingdom | 244             | 0.7336065573770490 | Brazil, France, United States        | 6.97%, 6.56%, 2.05%   |
| Singapore      | 70              | 0.7                | Taiwan, Malaysia, United States      | 10.0%, 5.71%, 2.86%   |
| North Dakota   | 980             | 0.6979591836734690 | Nebraska, Wyoming, South Dakota      | 4.49%, 3.57%, 3.37%   |
| Oklahoma       | 1012            | 0.6847826086956520 | Kansas, Texas, Arkansas              | 5.24%, 3.16%, 3.06%   |
| Brazil         | 255             | 0.6784313725490200 | Mexico, South Africa, United States  | 5.49%, 3.53%, 3.53%   |
| West Virginia  | 1044            | 0.6781609195402300 | Kentucky, Tennessee, Pennsylvania    | 4.69%, 3.93%, 3.35%   |
| Russia         | 173             | 0.6589595375722540 | Canada, Argentina, United States     | 15.03%, 2.89%, 2.31%  |
| Australia      | 170             | 0.6588235294117650 | Chile, South Africa, United States   | 10.59%, 9.41%, 2.35%  |
| Utah           | 1082            | 0.6515711645101660 | Idaho, Colorado, Wyoming             | 6.1%, 4.9%, 4.53%     |
| Nigeria        | 11              | 0.6363636363636360 | Russia, Indonesia, Brazil            | 18.18%, 9.09%, 9.09%  |
| Florida        | 996             | 0.6355421686746990 | South Carolina, Texas, Georgia       | 5.52%, 4.62%, 2.71%   |
| Arizona        | 1044            | 0.6350574712643680 | California, New Mexico, Nevada       | 13.6%, 10.73%, 2.39%  |
| Maine          | 1056            | 0.6325757575757580 | Rhode Island, Vermont, New Hampshire | 4.64%, 3.88%, 3.31%   |
| Vermont        | 976             | 0.6290983606557380 | Massachusetts, Maine, West Virginia  | 4.51%, 3.89%, 3.18%   |
| Oregon         | 904             | 0.6238938053097350 | Utah, Idaho, Washington              | 10.07%, 2.99%, 1.88%  |
| Thailand       | 109             | 0.6238532110091740 | Mexico, Brazil, United States        | 12.84%, 7.34%, 5.5%   |
| Israel         | 23              | 0.6086956521739130 | Peru, France, United States          | 8.7%, 4.35%, 4.35%    |
| Idaho          | 1104            | 0.605072463768116  | Montana, Utah, Wyoming               | 8.15%, 5.07%, 4.08%   |
| Montana        | 960             | 0.58125            | Utah, Idaho, Wyoming                 | 12.4%, 4.79%, 3.75%   |
| Mississippi    | 984             | 0.5792682926829270 | Alabama, Arkansas, Tennessee         | 7.62%, 7.01%, 3.25%   |

Figure 4: Main confusions for most accurate locations

|             | Number of items | Accuracy            | Top 3 most confused                   | Top 3 most confused %  |
|-------------|-----------------|---------------------|---------------------------------------|------------------------|
| Greece      | 21              | 0.14285714285714300 | Israel, Greece, Mexico                | 14.29%, 14.29%, 9.52%  |
| Hungary     | 15              | 0.13333333333333300 | Croatia, Hungary, Bulgaria            | 13.33%, 13.33%, 6.67%  |
| Switzerland | 15              | 0.13333333333333300 | Canada, Chile, Switzerland            | 13.33%, 6.67%, 6.67%   |
| Lithuania   | 18              | 0.05555555555555560 | United States, United Kingdom, Poland | 22.22%, 11.11%, 11.11% |
| Philippines | 19              | 0.05263157894736840 | Colombia, Indonesia, Japan            | 15.79%, 10.53%, 10.53% |
| Belgium     | 25              | 0.0                 | Poland, United States, United Kingdom | 20.0%, 16.0%, 12.0%    |
| Cambodia    | 10              | 0.0                 | United States, Mexico, Brazil         | 20.0%, 10.0%, 10.0%    |
| Croatia     | 12              | 0.0                 | South Korea, Finland, Italy           | 25.0%, 8.33%, 8.33%    |
| Czechia     | 24              | 0.0                 | Russia, Hungary, France               | 29.17%, 8.33%, 4.17%   |
| Denmark     | 16              | 0.0                 | Canada, United States, France         | 31.25%, 12.5%, 6.25%   |
| Latvia      | 14              | 0.0                 | Argentina, United States, Poland      | 21.43%, 21.43%, 7.14%  |
| Portugal    | 25              | 0.0                 | Japan, Italy, France                  | 20.0%, 16.0%, 8.0%     |
| Slovakia    | 14              | 0.0                 | Norway, Canada, Russia                | 14.29%, 14.29%, 7.14%  |
| Ukraine     | 11              | 0.0                 | Netherlands, Poland, United States    | 27.27%, 18.18%, 9.09%  |

Figure 5: Main confusions for least accurate locations

## 7.5 Top confusion for each class

Looking at Figure 4, we see that states are often confused for other states and countries are confused for other countries. This is due to the fact that these datasets are quite different from each other in terms of the watermarks they contain, essentially giving the model an easy cue to know if it is looking at a country or a state. We see that the errors being made the model seem to make sense. Singapore is often confused for Taiwan for example and Nevada is often confused for Utah.

Looking at figure 5 which shows the main confusion for the least accurate states, we see that the model performs poorly when we only have a limited amount of data, even to the point of getting a 0% accuracy.

## 8 Conclusion

In conclusion, we have made significant progress on the problem of image geolocation in the domain of the game Geoguessr. By utilizing more efficient models, incorporating pretraining, and using additional datasets, we were able to significantly improve upon previous work. Our best model, which uses EfficientNetB0, achieved an accuracy of 54.2%, outperforming the previous state-of-the-art model by more than 28% on top-1 accuracy and 26% on top-5 accuracy. We note that our final models can be considered undertrained given the fact that the training accuracies are still very close to the evaluation accuracies, indicating the potential for more performance through scaling the number of epochs.

## References

- [1] Geolocation - geoguessr images (50k). <https://www.kaggle.com/datasets/ubitquitin/geolocation-geoguessr-images-50k>.
- [2] adumb. Ai learns to play geoguessr. [www.youtube.com/watch?v=mM\\_dC1HVAQ4](http://www.youtube.com/watch?v=mM_dC1HVAQ4), Jul2020.
- [3] Anika Bahl. Csci 1430 final project report: Geoguessed, 2021.
- [4] Google. Street view static api overview, 2022.
- [5] James Hays and Alexei A. Efros. Im2gps: estimating geographic information from a single image. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2008.
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.
- [7] Andrew Healey. Geoguessing with deep learning, Mar 2021.
- [8] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows, 2021.
- [9] Eric Muller-Budack, Kader Pustu-Iren, and Ralph Ewerth. Geolocation estimation of photos using a hierarchical model and scene classification. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 563–579, 2018.
- [10] Shraman Pramanick, Ewa M Nowara, Joshua Gleason, Carlos D Castillo, and Rama Chellappa. Where in the world is this image? transformer-based geo-localization in the wild. In *European Conference on Computer Vision*, pages 196–215. Springer, 2022.
- [11] Sudharshan Suresh, Nathaniel Chodosh, and Montiel Abello. Deepgeo: Photo localization with deep neural network. *CoRR*, abs/1810.03077, 2018.
- [12] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. 2019.
- [13] Nirvan Theethira. Geoguessr ai: Image based geo-location, May 2022.
- [14] Nam Vo, Nathan Jacobs, and James Hays. Revisiting im2gps in the deep learning era. In *Proceedings of the IEEE international conference on computer vision*, pages 2621–2630, 2017.